CCE- Unit 4

I

Page No: 18
Date.

kth largest element in a Binary search Tree (BST)

problem statement

Develop a C++ program To find The kth largest element in a Binary search Tree (BST). The program accept a collection of distinct integer value. insert Them into a BST. takes the position k from The user. and finds The element at that position when The values are arranged in desending order. it must also check wheter The entered value of k is valid.

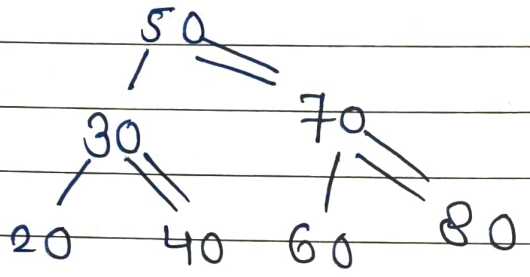
Introduction

A Binary search Tree is a non-linear hierarchical data structure used to store values in an organised form. it consist of nodes. where each node contains data and links To its left and right children. in an standard BST values smallar than a node are inserted into its subtree. while values greater Than The node are inserted into its right subtree. The same rule applies recursively To every subtree.

The kth largest element is The largest element is the value That appear at position k when all value are listed from largest to smallest.

Objective:- Construct a BST from input values. understand recursive Traversal Find a ranked element validate k. and analyse the The Time and space complexity of The solution.

BST diagram and example

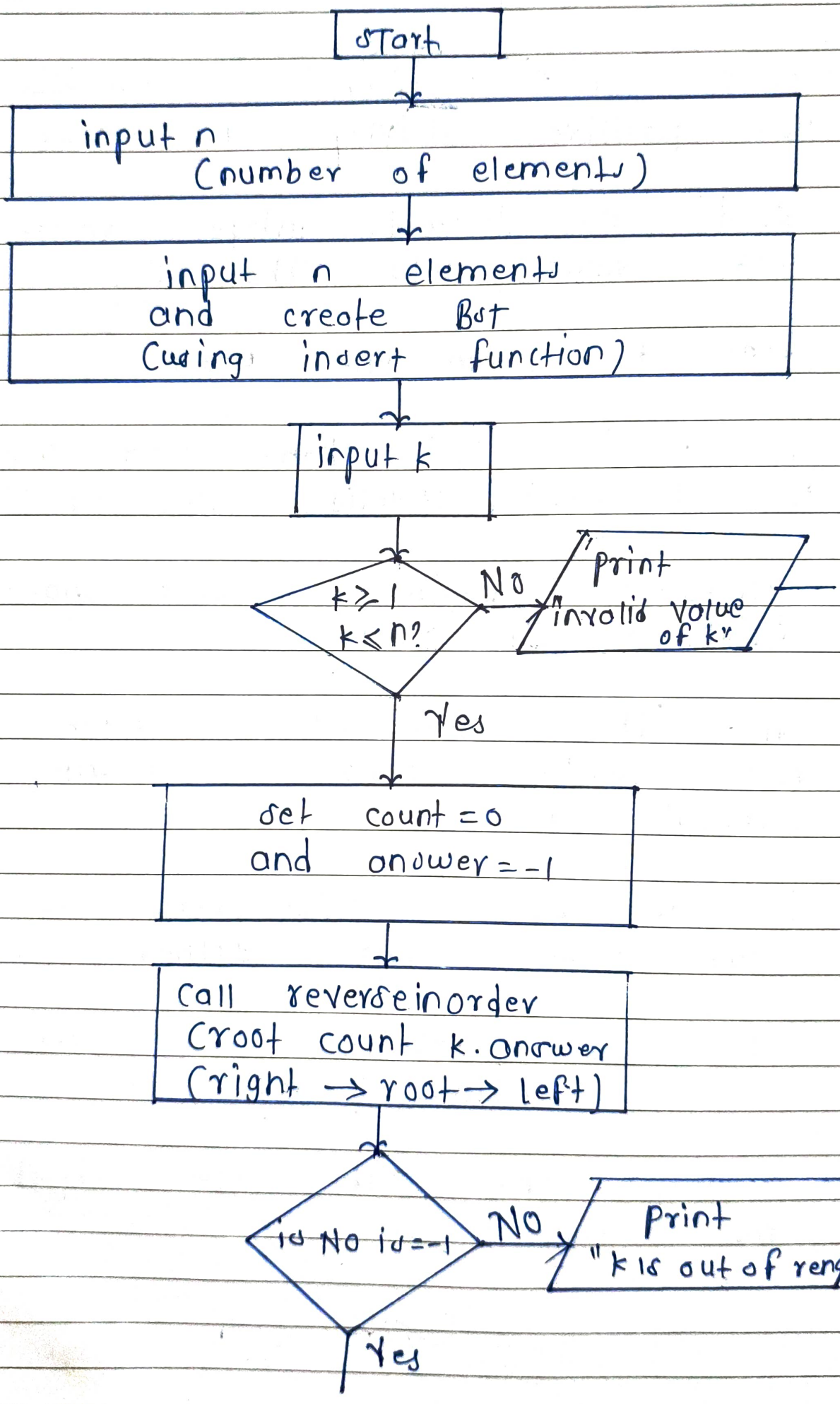


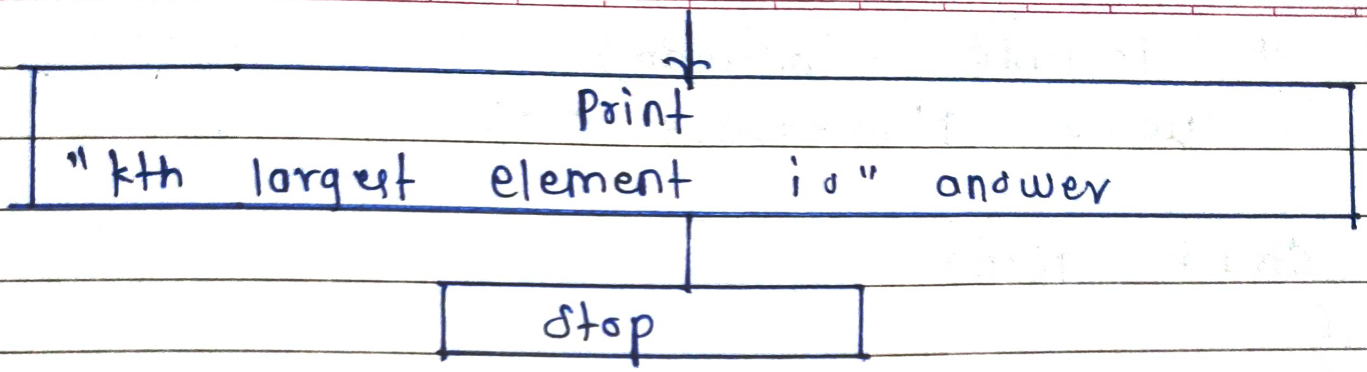
For The input values . 50, 30, 70, 20, 40, 60, 80 . The BST has 50 as its root. Values 30 and 20 are on The left side. while 70 and 60 are on the right side. Reverse inorder Traversal gives 80, 70, 60, 50, 40, 30, 20. If k=3 The Third value Visited is 60. so The Thired largest element is 60.

Algorithm

1. start The program
2. Read The Number of elements n. if n is not positive. display an error and stop.
3. Read n. distinct. integer values and insert each value into The BST.
4. Read k from The user.
5. if k is less Than 1 or greater Than n. display. invalid value of k''.
6. otherwise begins reverse inorder Traversal (right → root → left)
7. Visit the right subtree first. if k has not reached zero decrment k at The current node.
8. if k become zero. store The current Node value and stop further Traversal.
9. if The required Node has not been reach Continue into The left subtree.
10. Display The kth ~~req~~ largest element and stop.

flowchart


start
input n
(number of elements)
input n elements
and create BST
(using insert function)
input k
k>=1
k<=n?
No
print
"invalid value
of k"
Yes
set count=0
and answer=-1
call reverseinorder
(root count k.answer
(right → root → left))
is No is=-1
NO
Print
"k is out of range"
Yes


Print
"kth largest element is" answer
Stop

Page No:
Date.

Program

```
# include <iostream>
  using Nomespoce std:

struct Node:
{
   int data:
   Node* left. * right

   Node (int value)
{
   data = value:
   left ~ Right = NULL:
}
};

  Node * insert (Node * root ,int value)
{
   if ( root == NULL)
    return new Node (value):

if (value < root -> data)
   root -> left = insert (root -> left . value):
else if ( value > root ->data)
   root-> right =insert (root - >right . value);
```

```
    return root;
}

void findkthLargest (Node * root, int &k, int &no)
{
    if (root == NULL || k==0)
    return;

findkthLargest (root -> right, k, no);

if (k==0)
{
    no = root -> data;
    return ;
}

findkthlargest (root -> left, k, no);
}

int main()
{
    Node* root = NULL;
    int n, value, k, no =-1;

cout << " Enter Number of elements:";
cin >> n;
```

```
    if (n <= 0)
{
    cout << " Enter elements:";

    for( int i=0; i<n; i++)
{
    cin >> value;
    root = insert (root, value);
}
    cout << " invalid value of k";
    return 0;
}
    findkthLargest (root, k, and);

if (and != -1)
    cout << " kthlargest element is:" << and;
else
    cout << " Element not found ";

    return 0;
}
```

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node *left, *right;

    Node(int value)
    }
};

Node* insert(Node* root, int value)
{
    if (root == NULL)
        return new Node(value);

    if (value < root->data)
        root->left = insert(root->left, value);
    else if (value > root->data)
        root->right = insert(root->right, value);

    return root;
}

void findKthLargest(Node* root, int &k, int &ans)
{
    if (root == NULL || k == 0)
```

```
    return;

  findKthLargest(root->right, k, ans);

  if (k == 0)
    return;

  k--;

  if (k == 0)
  {
    ans = root->data;
    return;
  }

  findKthLargest(root->left, k, ans);
}

int main()
{
  Node* root = NULL;
  int n, value, k, ans = -1;

  cout << "Enter number of elements: ";
  cin >> n;

  if (n <= 0)
  {
```

```
    cout << "Invalid number of elements";
    return 0;
}

cout << "Enter elements: ";

for (int i = 0; i < n; i++)
{
    cin >> value;
    root = insert(root, value);
}

cout << "Enter K: ";
cin >> k;

if (k < 1 || k > n)
{
    cout << "Invalid value of K";
    return 0;
}

findKthLargest(root, k, ans);

if (ans != -1)
    cout << "Kth largest element is: " << ans;
else
    cout << "Element not found";
```